EECS348 Assignment 4 - Sudoku Solver System

UML Use-Case Analysis & Design Document
Object-Oriented C++ Implementation (Brute-Force Backtracking)
Author: Som Javia | Model Used: Grok AI

1. Use Case Identification

Use Case Name: Solve Sudoku Puzzles
Primary Actor: User
Supporting Actors: Puzzle Files, File System, Console Output
Scope: Stand-alone console Sudoku Solver application
Level: User-goal level
Related Classes (from implementation): Cell, Board, Solver, SudokuApplication

2. Brief Description

The User launches the Sudoku Solver application. The system automatically loads one or more Sudoku puzzles from predefined text files (puzzle1.txt … puzzle5.txt), where digits represent pre-filled cells and underscores represent empty cells. For each puzzle the system performs a depth-first search with backtracking to find all valid solutions (or determines that none exist). Results—including the original puzzle, every discovered solution, or the message “No solution found”—are printed to the console.

3. Step-by-Step Flow (Outline)

1. User starts the Sudoku Solver application (executes the program).
2. SudokuApplication initializes and obtains the list of puzzle file paths.
3. For each puzzle file:
4. a. Board loads the puzzle from the text file (digits and underscores).
5. b. A pristine copy of the original Board is retained for later display.
6. c. Solver receives the Board and begins the recursive backtracking search.
7. d. search() locates the next empty cell; if none remain, a complete solution is stored.
8. e. For each candidate digit 1–9 that is safe (row/column/block), the digit is placed and search recurses. **9.** f. On failure the digit is cleared (backtrack) and the next candidate is tried.
10. g. When the search finishes, all collected solutions (or zero) are available.
11. h. Results are formatted and written to Console Output: file name, original puzzle, solutions or “No solution found”.
12. After processing every file the application terminates.

4. Detailed Description

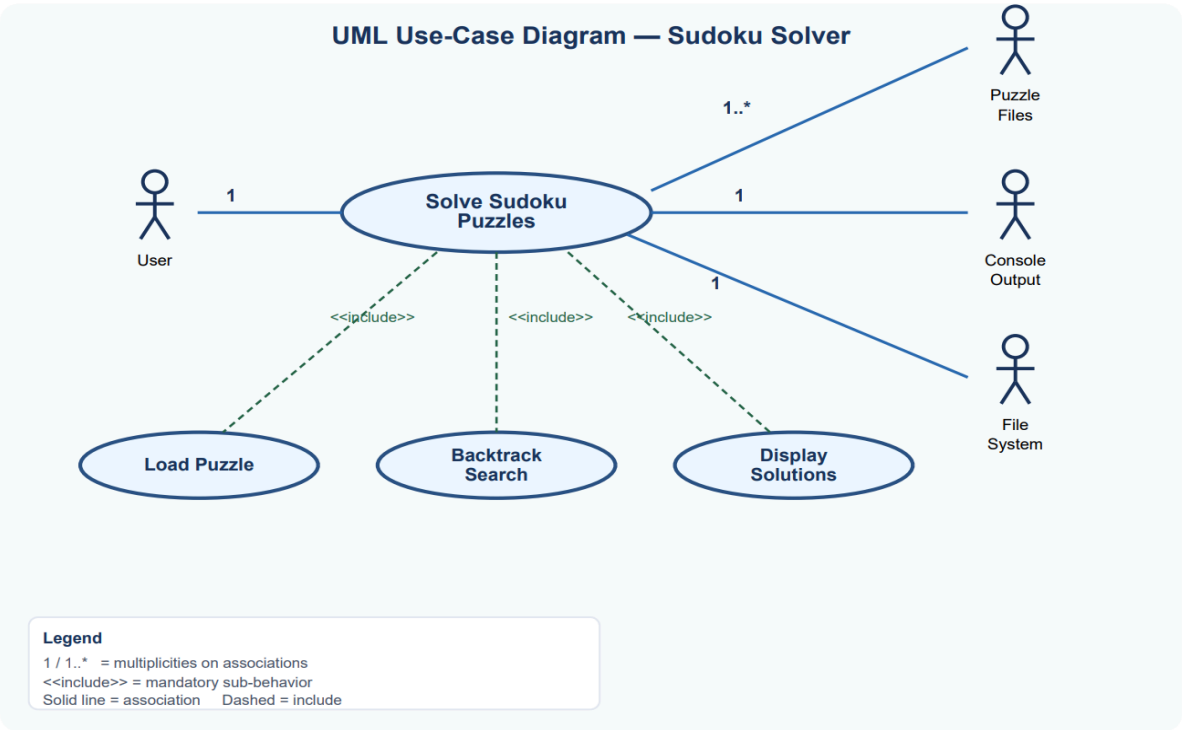
This use case realizes a classic exhaustive search for Sudoku solutions. The primary actor is the User who simply launches the program; no further interaction is required. Supporting actors are the Puzzle Files (the five .txt inputs), the File System (which supplies those files), and the Console Output (the destination of all diagnostic and solution text).

Internally the design follows a strict object-oriented decomposition: **Cell** encapsulates a single square, **Board** owns the 9×9 grid and provides loading, printing, safety checking and empty-cell location, **Solver** owns the recursive search and the collection of complete Boards, and **SudokuApplication** orchestrates the whole process. The search is pure depth-first with backtracking: at every empty cell the algorithm tries each legal digit, recurses, and restores the cell on return. Because the search continues after a solution is found, every valid filling of the board is reported. If the search tree is exhausted without a complete board, the system reports “No solution found”, thereby proving unsolvability for that puzzle.

Exception / edge paths: file-not-found or malformed input causes the affected puzzle to be skipped with an error message; an already inconsistent starting board (duplicate digits in a unit) naturally yields zero solutions; puzzles with multiple solutions are fully enumerated.

5. UML Use-Case Diagram

The diagram shows the primary use case **Solve Sudoku Puzzles** together with its supporting actors and the three mandatory included behaviors that correspond to the main stages of the algorithm.



Actors & Their Roles

Actor	Type	Role in the Use Case
User	Primary	Launches the application; receives all printed results
Puzzle Files	Supporting	Supply the five partially-filled Sudoku grids (.txt)
File System	Supporting	Provides read access to the puzzle text files
Console Output	Supporting	Receives formatted original puzzles and all solutions

Included Use Cases

• **Load Puzzle** — reads a .txt file and constructs a Board of Cell objects.
• **Backtrack Search** — the recursive depth-first search that explores candidate digits and collects complete solutions. • **Display Solutions** — formats and writes the original puzzle together with every solution (or the “No solution found” message) to the console.

6. Multiplicity & Design Rationale

The multiplicity **1** between User and Solve Sudoku Puzzles indicates that a single user invocation drives the entire batch of puzzles; the program is not multi-user. The multiplicity **1..*** toward Puzzle Files reflects that the application is designed to process a non-empty collection of puzzle files (five in the supplied implementation). Associations to File System and Console Output are both **1** because every run interacts with exactly one file system and writes to exactly one standard-output stream.

The three «include» relationships capture mandatory sub-behaviors that always occur as part of the main use case: a puzzle must be loaded, a search must be performed, and results must be displayed. This decomposition mirrors the object-oriented structure of the C++ code (Board::loadFromFile, Solver::search, Solver::printResults) and makes the algorithm's stages visible at the requirements level.

7. Mapping to Implementation Classes

Use-Case Element	C++ Class / Method
Solve Sudoku Puzzles	SudokuApplication::run()
Load Puzzle	Board::loadFromFile()
Backtrack Search	Solver::search() / Solver::solveAll()
Display Solutions	Solver::printResults() / Board::print()
Cell safety check	Board::isSafe()
Empty-cell location	Board::findEmpty()

This document provides the complete use-case view of the object-oriented Sudoku Solver. It links the external behavior visible to the User with the internal classes (Cell, Board, Solver, SudokuApplication) that realize the brute-force depth-first search with backtracking.